

Introduction to Python Programming

Chapter 12: Functions - Intermediate

Introduction

Now that you understand basic functions, it's time to explore more sophisticated techniques. This chapter covers how to call functions with named arguments, how to provide default values for parameters, and introduces the powerful concept of recursion. These tools will make your functions more flexible and capable of solving complex problems.

1. Positional vs Named Parameters

When calling functions, you can pass arguments by position or by name.

Positional Arguments (What You Already Know)

Arguments match parameters by order:

```
def introduce(name, age, city):  
    print(f"{name} is {age} years old and lives in {city}")  
  
introduce("Alice", 25, "Calgary")
```

Order matters! First argument → first parameter, second → second, etc.

Named Arguments (Keyword Arguments)

You can specify which parameter gets which value by name:

```
def introduce(name, age, city):  
    print(f"{name} is {age} years old and lives in {city}")  
  
introduce(name="Alice", age=25, city="Calgary")
```

Benefits of named arguments:

- Order doesn't matter!
- Makes code more readable
- Less prone to errors with many parameters

```
# These all work the same:
introduce(name="Bob", age=30, city="Toronto")
introduce(age=30, city="Toronto", name="Bob")
introduce(city="Toronto", name="Bob", age=30)
```

Mixing Positional and Named Arguments

You can mix both styles, BUT: **positional arguments must come first!**

```
def introduce(name, age, city):
    print(f"{name} is {age} years old and lives in {city}")

# Valid - positional first, then named
introduce("Alice", age=25, city="Calgary")
introduce("Bob", 30, city="Toronto")

# Invalid - named before positional
introduce(name="Alice", 25, "Calgary") # ERROR!
```

Rule: Once you use a named argument, all following arguments must be named.

Why Use Both?

Positional arguments:

- Quick and concise for simple functions
- Good when parameter order is obvious
- Common for functions with 1-3 parameters

Named arguments:

- Clearer for functions with many parameters
- Self-documenting code
- Prevents mistakes when parameters have similar types

```
# Positional - order matters, easy to mess up
create_rectangle(10, 20, "blue", "solid")

# Named - clear what each value means
create_rectangle(width=10, height=20, color="blue", style="solid")
```

Best practice: Use positional for simple cases, named for clarity with multiple parameters.

2. Optional Parameters with Default Values

Parameters can have default values, making them optional when calling the function.

Basic Default Parameters

```
def greet(name, greeting="Hello"):
    print(f"{greeting}, {name}!")

greet("Alice")           # Hello, Alice!
greet("Bob", "Good morning") # Good morning, Bob!
greet("Charlie", greeting="Hi") # Hi, Charlie!
```

How it works:

- `greeting` has default value `"Hello"`
- If you don't provide it, the default is used
- If you do provide it, your value overrides the default

Multiple Default Parameters

```
def create_user(username, email, role="user", active=True):
    print(f"Username: {username}")
    print(f>Email: {email}")
    print(f"Role: {role}")
    print(f"Active: {active}")

# Use all defaults
create_user("alice", "alice@email.com")

# Override some defaults
create_user("bob", "bob@email.com", role="admin")

# Override all defaults
create_user("charlie", "charlie@email.com", role="moderator", active=False)
```

CRITICAL RULE: Default Parameters Must Be Last

```
# Valid - defaults at the end
def calculate(a, b, c=10, d=20):
    return a + b + c + d

# Invalid - non-default after default
def calculate(a, b=10, c, d=20): # ERROR!
    return a + b + c + d
```

Why? Python reads parameters left to right. If a default parameter comes first, Python can't tell which argument goes where when you call the function.

Practical Example: Power Function

```
def power(base, exponent=2):  
    return base ** exponent  
  
print(power(5))           # 25 (5 squared - default exponent)  
print(power(5, 3))        # 125 (5 cubed)  
print(power(2, 10))       # 1024
```

Default of 2 makes sense because squaring is common!

3. Why Optional Parameters? (Function Overloading in Other Languages)

The Problem in Other Languages

Many programming languages (like Java, C++) use **function overloading** - defining multiple versions of the same function with different parameters:

```
// In Java - multiple versions of the same function  
public int calculate(int a, int b) {  
    return a + b;  
}  
  
public int calculate(int a, int b, int c) {  
    return a + b + c;  
}  
  
public int calculate(int a, int b, int c, int d) {  
    return a + b + c + d;  
}
```

You'd need three separate function definitions!

Python's Solution: Optional Parameters

Python doesn't have function overloading. Instead, we use optional parameters:

```
def calculate(a, b, c=0, d=0):
    return a + b + c + d

# Works like all three Java functions above:
calculate(5, 3)           # Like calculate(a, b)
calculate(5, 3, 2)        # Like calculate(a, b, c)
calculate(5, 3, 2, 1)      # Like calculate(a, b, c, d)
```

One function handles all cases! Much cleaner.

Another Example: Print Function

Python's built-in `print()` uses this extensively:

```
print("Hello")                # Uses default separator and end
print("Hello", "World")       # Uses default separator (space)
print("Hello", "World", sep="-") # Custom separator
print("Hello", end="")         # Custom end (no newline)
```

All from one function with optional parameters!

Benefits of This Approach

Compared to function overloading:

- Less code duplication
- Easier to maintain (one function, not many)
- More flexible - any combination of parameters
- Clearer intent with named arguments

```
def create_report(title, data, format="pdf", include_charts=True, color=True):
    # One function handles many combinations
    pass

# Customize exactly what you need
create_report("Sales", data, format="html", color=False)
```

4. Introduction to Recursion

Recursion is when a function calls itself. It's a powerful technique for solving problems that have a recursive structure.

The Concept

Think of looking in a mirror that reflects another mirror - you see infinite reflections. Each reflection shows a smaller version of the same thing. That's recursion!

In programming, recursion breaks a problem into smaller versions of the same problem.

Simple Example: Countdown

```
def countdown(n):  
    if n <= 0:  
        print("Done!")  
    else:  
        print(n)  
        countdown(n - 1) # Function calls itself!  
  
countdown(5)
```

Output:

```
5  
4  
3  
2  
1  
Done!
```

How it works:

1. `countdown(5)` prints 5, calls `countdown(4)`
2. `countdown(4)` prints 4, calls `countdown(3)`
3. `countdown(3)` prints 3, calls `countdown(2)`
4. `countdown(2)` prints 2, calls `countdown(1)`
5. `countdown(1)` prints 1, calls `countdown(0)`
6. `countdown(0)` prints "Done!" and stops (base case)

Critical Components of Recursion

Every recursive function needs:

1. **Base case** - condition that stops the recursion
2. **Recursive case** - function calls itself with modified input
3. **Progress toward base case** - each call gets closer to stopping

```
def countdown(n):
    # Base case - stops recursion
    if n <= 0:
        print("Done!")
    # Recursive case - calls itself
    else:
        print(n)
        countdown(n - 1) # Gets closer to base case
```

Without a base case, the function would call itself forever (infinite recursion)!

5. Fibonacci Sequence with Recursion

The Fibonacci sequence is a classic example of recursion: each number is the sum of the previous two.

Sequence: 0, 1, 1, 2, 3, 5, 8, 13, 21, 34...

Pattern:

- Position 0: 0
- Position 1: 1
- Position n: fibonacci(n-1) + fibonacci(n-2)

Recursive Fibonacci

```
def fibonacci(n):
    # Base cases
    if n == 0:
        return 0
    if n == 1:
        return 1

    # Recursive case
    return fibonacci(n - 1) + fibonacci(n - 2)

# Get specific Fibonacci numbers
print(fibonacci(0))    # 0
print(fibonacci(1))    # 1
print(fibonacci(5))    # 5
print(fibonacci(10))   # 55
```

How fibonacci(5) works:

```

fibonacci(5)
= fibonacci(4) + fibonacci(3)
= (fibonacci(3) + fibonacci(2)) + (fibonacci(2) + fibonacci(1))
= ((fibonacci(2) + fibonacci(1)) + (fibonacci(1) + fibonacci(0))) + ((fibonacci(1) +
fibonacci(0)) + 1)
= ((1 + 1) + (1 + 0)) + ((1 + 0) + 1)
= (2 + 1) + (1 + 1)
= 3 + 2
= 5

```

Building Fibonacci Rows

```

def print_fibonacci_sequence(count):
    for i in range(count):
        print(f"Position {i}: {fibonacci(i)}")

print_fibonacci_sequence(10)

```

Output:

```

Position 0: 0
Position 1: 1
Position 2: 1
Position 3: 2
Position 4: 3
Position 5: 5
Position 6: 8
Position 7: 13
Position 8: 21
Position 9: 34

```

Understanding the Recursion

For `fibonacci(4)`:

```

          fib(4)
         /    \
      fib(3)  fib(2)
     /  \    /  \
  fib(2) fib(1) fib(1) fib(0)
   /  \   |    |    |
 fib(1) fib(0) 1    1    0
   |    |
   1    0

```


Each call branches into two more calls until reaching base cases (0 or 1).

Recursion vs Loops

You could also calculate Fibonacci with a loop:

```
def fibonacci_loop(n):  
    if n == 0:  
        return 0  
    if n == 1:  
        return 1  
  
    prev2 = 0  
    prev1 = 1  
  
    for i in range(2, n + 1):  
        current = prev1 + prev2  
        prev2 = prev1  
        prev1 = current  
  
    return prev1
```

Recursive version:

- Elegant and mirrors the mathematical definition
- Easier to understand the concept
- Can be slower for large numbers (recalculates same values)

Loop version:

- More efficient (calculates each value once)
- More code to write
- Less intuitive

When to use recursion:

- Problem naturally breaks into smaller versions of itself
- Code clarity is more important than performance
- Working with tree-like or nested structures

6. More Recursion Examples

Factorial

```
def factorial(n):
    # Base case
    if n == 0 or n == 1:
        return 1

    # Recursive case
    return n * factorial(n - 1)

print(factorial(5)) # 120 (5 * 4 * 3 * 2 * 1)
```

How it works:

```
factorial(5)
= 5 * factorial(4)
= 5 * (4 * factorial(3))
= 5 * (4 * (3 * factorial(2)))
= 5 * (4 * (3 * (2 * factorial(1))))
= 5 * (4 * (3 * (2 * 1)))
= 5 * 4 * 3 * 2 * 1
= 120
```

Sum of Numbers

```
def sum_to_n(n):
    # Base case
    if n == 0:
        return 0

    # Recursive case
    return n + sum_to_n(n - 1)

print(sum_to_n(5)) # 15 (5 + 4 + 3 + 2 + 1 + 0)
```

Power Function

```
def power(base, exponent):
    # Base case
    if exponent == 0:
        return 1

    # Recursive case
    return base * power(base, exponent - 1)

print(power(2, 5)) # 32 (2 * 2 * 2 * 2 * 2)
```

7. Practical Examples

Example 1: Flexible Greeting

```
def greet(name, greeting="Hello", punctuation="!"):
    return f"{greeting}, {name}{punctuation}"

print(greet("Alice"))           # Hello, Alice!
print(greet("Bob", "Good morning")) # Good morning, Bob!
print(greet("Charlie", punctuation=".")) # Hello, Charlie.
print(greet("Diana", greeting="Hi", punctuation="!!!")) # Hi, Diana!!!
```

Example 2: Calculate Total with Optional Tax

```
def calculate_total(price, quantity=1, tax_rate=0.05):
    subtotal = price * quantity
    tax = subtotal * tax_rate
    total = subtotal + tax
    return total

print(calculate_total(10))           # $10.50 (1 item, 5% tax)
print(calculate_total(10, 3))        # $31.50 (3 items, 5% tax)
print(calculate_total(10, 3, 0.10))  # $33.00 (3 items, 10% tax)
print(calculate_total(10, quantity=2, tax_rate=0.08)) # $21.60
```

Example 3: List Sum (Recursive)

```
def sum_list(numbers):
    # Base case - empty list
    if len(numbers) == 0:
        return 0

    # Recursive case - first number + sum of rest
    return numbers[0] + sum_list(numbers[1:])

print(sum_list([1, 2, 3, 4, 5])) # 15
```

How it works:

```
sum_list([1, 2, 3, 4, 5])
= 1 + sum_list([2, 3, 4, 5])
= 1 + (2 + sum_list([3, 4, 5]))
= 1 + (2 + (3 + sum_list([4, 5])))
= 1 + (2 + (3 + (4 + sum_list([5]))))
= 1 + (2 + (3 + (4 + (5 + sum_list([])))))
= 1 + (2 + (3 + (4 + (5 + 0))))
= 15
```

8. Common Pitfalls and Best Practices

Pitfall 1: Mutable Default Arguments

Danger:

```
def add_item(item, items=[]): # DON'T DO THIS!
    items.append(item)
    return items

print(add_item("apple")) # ['apple']
print(add_item("banana")) # ['apple', 'banana'] - Unexpected!
```

The default list is shared across calls!

Solution:

```
def add_item(item, items=None):
    if items is None:
        items = []
    items.append(item)
    return items

print(add_item("apple")) # ['apple']
print(add_item("banana")) # ['banana'] - Correct!
```

Pitfall 2: Infinite Recursion

```
def countdown(n):
    print(n)
    countdown(n - 1) # No base case - runs forever!
```

Always have a base case!

Best Practices

1. **Parameter Order:**

- Required parameters first
- Optional parameters last
- Most important parameters first

2. **Default Values:**

- Use immutable defaults (numbers, strings, None)
- Never use mutable defaults (lists, dicts)

3. **Recursion:**

- Always have a clear base case
- Ensure progress toward base case
- Consider iterative alternatives for performance

4. **Documentation:**

- Document what parameters mean
- Explain default values
- Show example usage

Key Takeaways

- ✓ **Positional arguments** match by order, **named arguments** match by name
 - ✓ **Named arguments** must come after positional arguments
 - ✓ **Default parameters** make arguments optional
 - ✓ **Default parameters must be last** in the parameter list
 - ✓ **Python uses defaults instead of function overloading** - cleaner than multiple function versions
 - ✓ **Recursion** is when a function calls itself
 - ✓ **Every recursive function needs:**
 - Base case (stops recursion)
 - Recursive case (calls itself)
 - Progress toward base case
 - ✓ **Fibonacci** is a classic recursion example: each number = sum of previous two
 - ✓ **Never use mutable defaults** (use None, then create inside function)
 - ✓ **Recursion vs loops** - recursion is elegant but can be less efficient
-

Reflection Questions

1. What's the difference between positional and named arguments?
2. Why must positional arguments come before named arguments?
3. Why must default parameters be last in the parameter list?
4. How does Python's approach differ from function overloading?
5. What three components does every recursive function need?
6. What happens without a base case in recursion?
7. How does Fibonacci demonstrate recursion?
8. Why should you avoid mutable default arguments?

Practice Exercises

1. **Flexible Calculator:** Create a function that adds two numbers with optional third and fourth numbers (defaults to 0).
2. **Format Name:** Create a function that formats a name with optional title (default "Mr./Ms.") and suffix (default "").
3. **Countdown Function:** Write a recursive function that counts down from N to 1.
4. **Recursive Sum:** Write a recursive function that sums all numbers from 1 to N.
5. **Power Function:** Create a recursive power function ($\text{base}^{\text{exponent}}$).
6. **Fibonacci List:** Create a function that returns a list of the first N Fibonacci numbers.
7. **Search Function:** Create a recursive function that searches for a value in a list.
8. **Restaurant Bill:** Create a function with price, tip (default 15%), tax (default 5%), and split (default 1 person).
9. **Recursive Maximum:** Write a recursive function that finds the maximum value in a list.
10. **String Reverser:** Write a recursive function that reverses a string.